

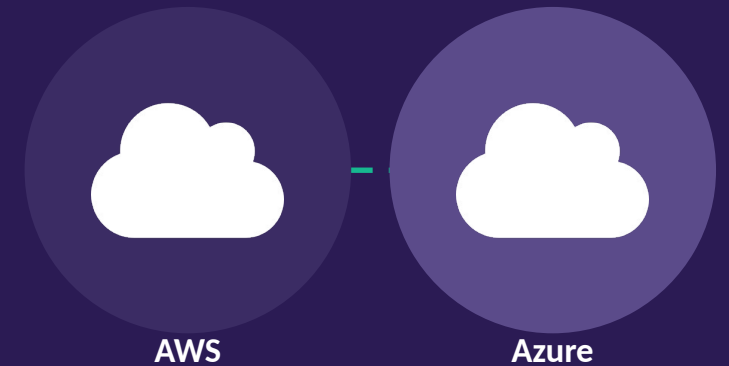
PROJECT 6

Multi-Cloud Setup AWS & Azure using Terraform

One Terraform Codebase, Redundant Infrastructure Across Two Clouds



Instructor Deck • Terraform Fundamentals Series



SCENARIO

Deploy the Same Infrastructure in Both Clouds

You need redundant infrastructure across AWS and Azure — if one cloud has an outage, the other keeps serving traffic.



AWS

- ✓ EC2 — virtual machines
- ✓ RDS (MySQL) — managed database
- ✓ ALB — application load balancer



Azure

- ✓ Virtual Machines — compute
- ✓ PostgreSQL — managed database
- ✓ Load Balancer — traffic distribution

Both are declared and managed from one Terraform codebase using the AWS and AzureRM providers.

STEP 1

Configure Multi-Cloud Terraform Providers

```
provider "aws" {  
    region = "us-east-1"  
}  
  
provider "azurerm" {  
    features {}  
}
```



Defines AWS & Azure providers.

Every resource block below will be prefixed `aws_` or `azurerm_` — Terraform routes each to the right cloud automatically based on its provider.

Credentials for each provider are typically supplied via environment variables (e.g. `AWS_ACCESS_KEY_ID`, `ARM_CLIENT_ID`) — never hard-coded in `.tf` files.

Deploy Infrastructure in Each Cloud

Step 2: AWS — EC2 instance

```
resource "aws_instance" "aws_vm" {
  ami          = "ami-123456"
  instance_type = "t3.medium"
}
```

Step 3: Azure — Virtual Machine

```
resource "azurerm_virtual_machine" "azure_vm" {
  name          = "azure-vm"
  location      = "East US"
  resource_group_name =
    azurerm_resource_group.rg.name
  vm_size      = "Standard_DS1_v2"
}
```

Same intent, different provider syntax

Compute resource	aws_instance	azurerm_virtual_machine
Sizing parameter	instance_type	vm_size
Placement	AMI + region	location + resource group

STEP 4

Configure a Global Load Balancer

A **Route 53 alias record** maps one global DNS name to infrastructure spanning both clouds.

```
resource "aws_route53_record" "multi_cloud_lb" {
  zone_id = var.zone_id
  name     = "app.example.com"
  type     = "A"
  alias {
    name = aws_lb.app_lb.dns_name
    evaluate_target_health = true
  }
}
```



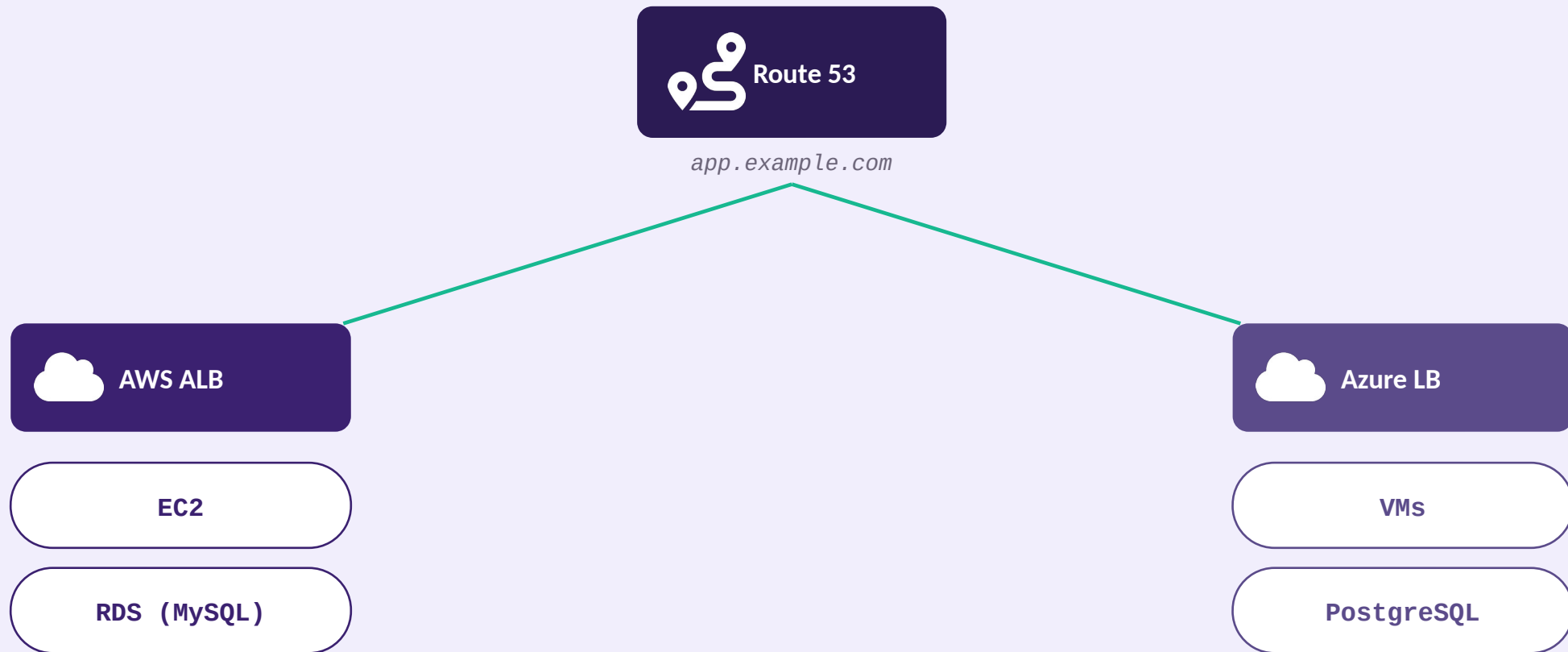
Maps global DNS to AWS & Azure

app.example.com

`evaluate_target_health = true`

Route 53 continuously checks endpoint health and can fail traffic over between clouds.

How Traffic Reaches Both Clouds



If AWS becomes unhealthy, Route 53 fails traffic over to Azure — and vice versa.

Confirm Everything Deployed Correctly

1

Check AWS VM & RDS

```
$ aws ec2 describe-instances
```

2

Check Azure VM

```
$ az vm list
```

3

Test Global Load Balancer

```
$ nslookup app.example.com
```

What We Have Built



AWS & Azure providers configured in one codebase



EC2 instance in AWS and Virtual Machine in Azure



RDS (MySQL) in AWS and PostgreSQL in Azure



Route 53 global DNS record routing to both clouds



Result: redundant infrastructure across two clouds, with automatic DNS failover.